

# DocLens: Developer & Enterprise API Integration Manual

Complete RESTful API Reference, Endpoint Schemas & Integration Guidelines

Document Standard: Devpost Hackathon Submission Artifact | Author: DocLens Engineering Team

---

## 1. API Overview & Technical Standards

---

The DocLens API is a high-performance RESTful web service built with Python 3.11 and FastAPI. All API requests and responses communicate using UTF-8 encoded JSON payloads or multipart/form-data for document file transfers. The API versioning prefix is /api/v1.

### 1.1 Base URL & Environment Endpoints

Production Base URL: `https://doclens.yvesseraphin.xyz/api/v1`

Local Development URL: `http://localhost:8000/api/v1`

### 1.2 Authentication Headers

All protected API routes require HTTP Bearer Token authentication. Authentication JWTs are issued via Supabase Auth upon successful login:

Authorization: Bearer <jwt\_access\_token>

## 2. Service Health & System Status Endpoint

---

### 2.1 GET /api/v1/health

Checks service operational status, database connectivity, and PyTorch model readiness.

- **Request Method:** GET
- **Authentication Required:** None (Public Endpoint)
- **Response 200 OK Payload:** {  
 "status": "healthy",  
 "version": "1.0.0",  
 "pytorch\_device": "cpu",  
 "database": "connected"  
}

## 3. Authentication & Account Management Endpoints

---

### 3.1 POST /api/v1/auth/register

Registers a new user account.

- **Request Body:** {  
 "email": "user@example.com",  
 "password": "SecurePassword123!"  
}
- **Response 201 Created:** {  
 "user\_id": "8f3b2a1c-...",  
 "email": "user@example.com",  
 "message": "Registration successful"  
}

### 3.2 POST /api/v1/auth/login

Authenticates credentials and returns a JWT access token.

- **Request Body:** {  
 "email": "user@example.com",  
 "password": "SecurePassword123!"  
}
- **Response 200 OK:** {  
 "access\_token": "eyJhbGciOi...",  
 "token\_type": "bearer",  
 "expires\_in": 3600,  
 "user": { "id": "8f3b2a1c-...", "email": "user@example.com" }  
}

### 3.3 GET /api/v1/auth/me

Retrieves authenticated user profile information.

- **Headers:** Authorization: Bearer <jwt\_token>
- **Response 200 OK:** {  
 "id": "8f3b2a1c-...",  
 "email": "user@example.com",  
 "created\_at": "2026-08-08T09:00:00Z"  
}

### 3.4 POST /api/v1/auth/forgot-password

Initiates password recovery email workflow.

- **Request Body:** {  
 "email": "user@example.com"  
}
- **Response 200 OK:** {  
 "message": "Password recovery email dispatched"  
}

## 4. Case Workspace Management Endpoints

### 4.1 GET /api/v1/cases

Lists all forensic cases owned by the authenticated user.

- **Headers:** Authorization: Bearer <jwt\_token>
- **Response 200 OK:** [  
 {  
 "id": "9b1deb4d-...",  
 "title": "Contract Audit #4092",  
 "signer\_name": "John Doe",  
 "status": "analyzed",  
 "created\_at": "2026-08-08T10:15:00Z"  
 }  
]

### 4.2 POST /api/v1/cases

Creates a new case workspace.

- **Request Body:** {  
  "title": "Wire Transfer Audit #102",  
  "signer\_name": "Jane Smith"  
}
- **Response 201 Created:** {  
  "id": "c4a2e1b9-...",  
  "title": "Wire Transfer Audit #102",  
  "signer\_name": "Jane Smith",  
  "status": "draft"  
}

4.3 DELETE /api/v1/cases/{case\_id}

Deletes a case workspace and all associated documents and storage objects.

- **Response 200 OK:** {  
  "message": "Case c4a2e1b9-... deleted successfully"  
}

5. Core Analysis Endpoint: POST /api/v1/analyze

Performs end-to-end automated signature verification on uploaded materials.

- **Content-Type:** multipart/form-data
- **Form Parameter case\_id:** UUID string of target case workspace.
- **Form Parameter questioned\_file:** Questioned document file (JPEG, PNG, WebP, BMP, or PDF).
- **Form Parameter reference\_files:** 1 to 5 genuine reference specimen files.

5.1 Sample Analysis Response JSON Schema

```
{  
  "status": "success",  
  "case_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",  
  "verdict": "AUTHENTIC",  
  "confidence_score": 0.9642,  
  "deep_similarity": 0.9710,  
  "structural_score": 0.8350,  
  "cedar_threshold": 0.9271469,  
  "features": {  
    "stroke_width_variation": 0.042,  
    "letter_proportion_inconsistency": 0.051,  
    "baseline_deviation_px": 2.14,  
    "pen_lift_matches": 4,  
    "slant_deviation_deg": 1.8  
  },  
  "narrative_report": "The questioned signature demonstrates strong structural alignment..."  
}
```

6. Error Handling & HTTP Status Reference

| HTTP Status Code | Error Name | Description / Common Triggers |
|------------------|------------|-------------------------------|
|------------------|------------|-------------------------------|

|                           |                               |                                                                           |
|---------------------------|-------------------------------|---------------------------------------------------------------------------|
| 400 Bad Request           | Invalid Multipart Payload     | Missing reference files or unparseable upload payload format.             |
| 401 Unauthorized          | Missing/Expired Bearer Token  | Missing or invalid JWT token in HTTP Authorization header.                |
| 403 Forbidden             | Resource Access Denied        | Attempting to access a case owned by another user ID.                     |
| 404 Not Found             | Case ID Not Found             | Target case_id UUID does not exist in PostgreSQL database.                |
| 413 Payload Too Large     | File Size Limit Exceeded      | Uploaded file size exceeds the 10 MB system limit.                        |
| 422 Unprocessable         | Image Safety Cap Exceeded     | Decoded image exceeds 20M pixels or 40 PDF page cap.                      |
| 500 Internal Server Error | Pipeline Processing Exception | Unexpected exception in CV pipeline; returns safe fallback error message. |

## 7. Enterprise Code Integration Examples

### 7.1 Python Integration Example (httpx)

```
import httpx
```

```
headers = {"Authorization": "Bearer YOUR_JWT_TOKEN"}
files = [
    ("questioned_file", ("questioned.png", open("questioned.png", "rb"), "image/png")),
    ("reference_files", ("ref1.png", open("ref1.png", "rb"), "image/png"))
]
data = {"case_id": "YOUR_CASE_UUID"}
response = httpx.post("https://doclens.yvesseraphin.xyz/api/v1/analyze", data=data, files=files,
headers=headers)
print(response.json())
```

### 7.2 JavaScript Fetch Integration Example

```
const formData = new FormData();
formData.append("case_id", caseId);
formData.append("questioned_file", questionedFileInput.files[0]);
formData.append("reference_files", ref1FileInput.files[0]);

const res = await fetch("https://doclens.yvesseraphin.xyz/api/v1/analyze", {
  method: "POST",
  headers: { "Authorization": `Bearer ${token}` },
  body: formData
});

const data = await res.json();
console.log("Analysis Result:", data);
```

### 7.3 cURL Command Integration Example

```
curl -X POST "https://doclens.yvesseraphin.xyz/api/v1/analyze" \
-H "Authorization: Bearer YOUR_JWT_TOKEN" \
-F "case_id=YOUR_CASE_UUID" \
```

```
-F "questioned_file=@questioned.pdf" \  
-F "reference_files=@reference1.png"
```

---

## 8. Rate Limiting & Safety Throttling Policy

---

To protect backend inference resources from denial-of-service vectors, the API enforces a rate limit of 60 requests per minute per IP address on public endpoints and 120 requests per minute on authenticated user endpoints.

---

## 9. Webhook & Asynchronous Event Notifications

---

For high-volume enterprise integrations processing batch contract verification, DocLens supports asynchronous webhooks. When an analysis completes, the API posts an HTTP POST callback payload containing complete analysis JSON results to configured subscriber URLs.

---

## 10. Pydantic Request/Response Data Validation Schemas

---

The backend API uses Pydantic v2 schemas to ensure strict data validation and automatic OpenAPI documentation generation. All models enforce type safety, UUID formatting, non-empty string constraints, and score floating-point range bounds.

---

## 11. Security Best Practices & CORS Configuration

---

Cross-Origin Resource Sharing (CORS) is explicitly configured to permit requests from authorized origin domains. HTTPS TLS 1.3 encryption is enforced on all endpoints in production.